

**Embedded Linux
Final Mini Project
Group 17**



**Lars Panduro Bunszey, labun19@student.sdu.dk
Jonas Finne Skovbakke, jskov19@student.sdu.dk
Adam Kolodziej, adkol23@student.sdu.dk**

Github: <https://github.com/u000000/EMLI-mini-project>

Embedded Linux Spring 2024
University of Southern Denmark (SDU)
Odense
June 2024

1 Solution approach

1.1 Raspberry Pi Wildlife camera

The Raspberry Pi 4 (RPi) is running Raspberry Pi OS and is set up as an access point. The approach to taking photos on the wildlife camera is to utilize a shell script "take_photo.sh", which is responsible for taking a photo. This script can be called with different parameters depending on how the photo should be saved and what the trigger is. As seen in figure 1 the 3 ways used to take a photo is by motion detection controlled by the "motion_detect.sh" script, an external trigger controlled by the "external_trigger.sh" script, and every 5 minutes using a cronjob.

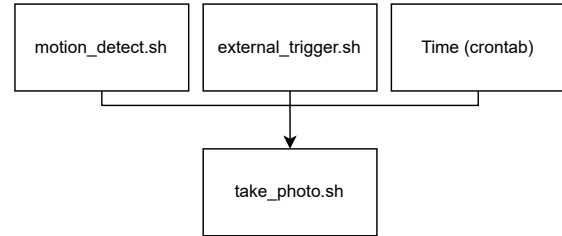


Figure 1: Diagram of taking photos

The external trigger is using an ESP8266 microcontroller. This is connected to the RPi access point and has a pressure plate connected. When the pressure plate is triggered a message is published to an MQTT topic and a subscriber is located in the "external_trigger.sh" from before.

1.1.1 Rain detector

To solve the issue of possible rain obscuring the image by leaving raindrops on the lens, a Raspberry Pi Pico (Pico) is running a script that writes to the wildlife camera over a serial connection every 0.1 s. The message from the Pico tells the position of the wiper motor, and whether it is raining or not. This will be read by one script, that is publishing if it is raining to a MQTT topic and another script will subscribe to it and publish to another MQTT topic what the desired angle of the wiper motor should be. Then the first script will subscribe to the result and write this to the serial port using the JSON format required by the Pico to make it move the servo motor to a specific position. The setup is as seen in figure 2.

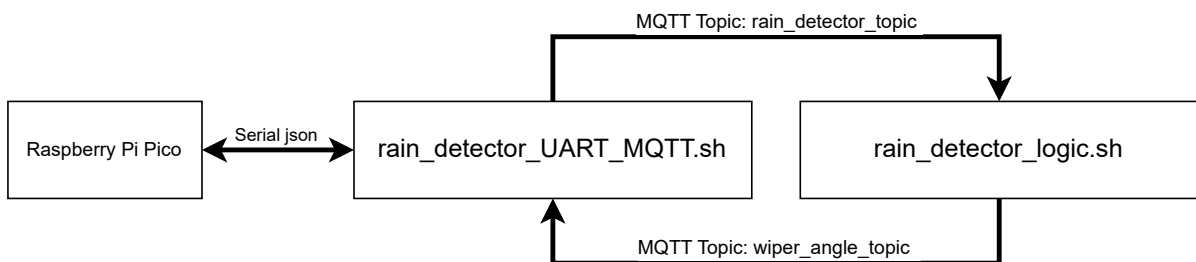


Figure 2: Diagram of detection of rain

1.1.2 Local website

A local website is set up using the Apache2 web server. This is set up to show images, sidecar JSON files, and a log file for all relevant events, such as if it has started and stopped raining, a photo is taken using a specific trigger, and which files have been transferred to a drone.

1.1.3 Power failure recovery

To make sure everything is running and fully operational after a power failure, all scripts on the RPi are set up to be running as services.

1.2 Drone

Each drone will have a static IP in the range of 192.168.10.X. This makes it possible to identify the drone id based on the IP. When the drone comes close to the RPi it will automatically connect to its access point. When the drone is connected to a WiFi point, the drone has a NTP server, that the camera can synchronize its time with. While the drone is connected to the RPi it will also log WiFi link quality for later flight path optimization, in a local MySQL database. The WiFi link quality is important for receiving pictures from the camera. In figure 3 it is shown that every time the camera takes a photo it is stored in an images directory. At the same time it creates a symbolic link to the photo in two other directories, which are used as databases. When the drone is connected it will try to grab as many photos as possible from the webcam folder using *rsync* and a script on the RPi will check what has been grabbed by checking the difference and adding to each sidecar file which drone has collected the corresponding photo and delete them from the log database.

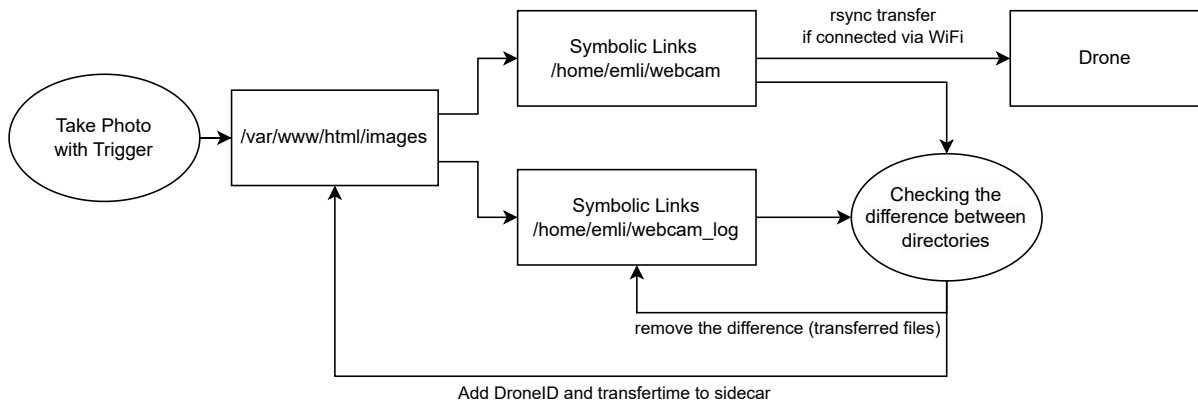


Figure 3: Diagram of the drone transfer system

1.3 Cloud

The photos will be manually moved from the drone's SD card to an offload directory on a PC. A script is then run which performs annotation of the images by use of *Ollama*, which is a local LLM. Afterward, the JSON sidecar files are updated before moving all files to the cloud and pushing the sidecar files to GitHub.

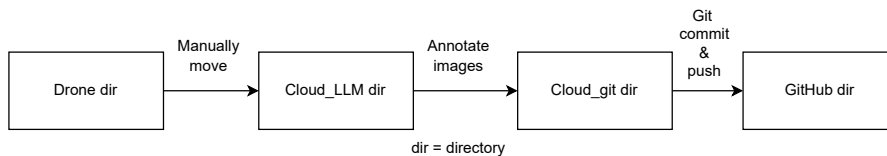


Figure 4: Diagram of cloud

1.4 Security

To make the connection between the Drone and the camera secure, a SSH key is used so the password is not stored in plain text. This together with a strong password and not using the default username *pi* ensures more security. The RPi is also setup to ban IP addresses after 3 failed SSH login attempts and the *nftables* set up to only allow the ports used for SSH and HTTP.

2 Solution Description

2.1 Raspberry Pi Wildlife camera

The wildlife camera is running on a RPi running Raspberry Pi OS. This section describes the bash scripts used to do the tasks required for the wildlife camera to work on the savanna.

2.1.1 Camera

To take a photo using the RPi Camera, a script called "take_photo.sh" has been created. This has been created to take in 5 parameters:

```
1 TRIGGER=${1:-"External"}
2 SAVEPATH=${2:-"/var/www/html/images/"}
3 SUBFOLDERDATE=${3:-true}
4 SYMLINK=${4:-"/home/emli/webcam/"}
5 SYMLINKLOG=${5:-"/home/emli/webcam/.log"}
```

The first two parameters are the trigger and save location. The third parameter is if the image should be saved in a subfolder with the current date. The fourth and fifth parameter are the locations of symbolic links which is explained in section 2.2.2. To take an image *rpicas* is used:

```
rpicas-jpeg -o $FOLDER$IMAGENAME$IMAGETYPE --width 640 --height 480 -t 1
```

and to get the data for the sidecar file the *exiftool* is used together with the variable needed, piped into *awk* to get the fourth word for distance as this:

```
1 exiftool -SubjectDistance $FOLDER$IMAGENAME$IMAGETYPE | awk '{print $4}'
```

Then appending each line of the JSON to the sidecar file:

```
1 #Make a JSON
2 echo "{" > $FOLDER$IMAGENAME$SIDECARTYPE
3 echo "  \"File Name\": \"$IMAGENAME$IMAGETYPE\"," >>
4   $FOLDER$IMAGENAME$SIDECARTYPE
5   ..., ..., ...,
6 echo "  \"ISO\": $ISO" >> $FOLDER$IMAGENAME$SIDECARTYPE
7 echo "}" >> $FOLDER$IMAGENAME$SIDECARTYPE
```

2.1.2 External trigger

The external trigger is powered by an ESP8266, which is running a MQTT publisher, that publishes "triggered" to the topic "emli/trigger/pressure_plate" when something steps on the external pressure plate. A script called "external_trigger.sh" is running as a service, which has a subscriber to the same topic, using the *mosquitto* library:

```
mosquitto_sub -h localhost -u emli -P raspberry -t "emli/trigger/pressure_plate" | while read TRIGGER
```

when the message then contains the string "triggered" a photo is taken using the script in section 2.1.1 as:

```
/path/to/take_photo.sh External /var/www/html/images/ true /home/emli/webcam/ /home/emli/webcam.log/
```

2.1.3 Motion detector

The motion detection is using the "take_photo.sh" describer in section 2.1.1 aswell. This uses it with only parameters being used: Trigger=Motion Location="/home/emli/webcam-temp/" subfolderDate=false. This is done because the files need to be compared before they get added to the database. The *.jpg files are found by saying:

```
jpeg_files=$(ls $directory | grep .jpg | sort -r)
```

This reads the list of files in the directory, piped into a filter to only get the lines containing .jpg and then sorting them by when they are added, newest first. Then the 2 newest images are found and compared using the python script:

```
image1=$(echo $jpeg_files | awk '{print $1}')
image2=$(echo $jpeg_files | awk '{print $2}')
output=$(python3 /home/emli/EMLI-mini-project/src/RPi/Camera/motion_detect.py $image1 $image2)
```

image2 is then deleted, because this is no longer needed. If motion is detected between the two photos image1 is copied to the webserver location and symbolic links are created as explained in section 2.2.2.

2.1.4 Rain detector

The rain detector is using the Pico and two scripts running on the main RPi as seen in figure 2. The Pico is connected via USB to the RPi and communicates using a serial port. The two RPi bash scripts communicate with each other via MQTT.

The Pico publishes JSON data every cycle:

```
{"wiper_angle": 0, "rain_detect": 0}
```

It takes an input message with the wiper angle (0-180):

```
{"wiper_angle": 180}
```

If a wrong number is sent, the Pico sends back:

```
{"serial": "angle_error"}
```

If there is an issue with the data sent, Pico will send back the following message:

```
{"serial": "json_error"}.
```

rain_detector_logic.sh This script is responsible for preparing the data about the wiper angle which will be sent to the Pico. It has a MQTT subscriber using the *mosquitto* library, which receives messages from the other bash script running on the RPi - on the topic called *rain_detector_topic*. If the message says 'RAINING', the script publishes the desired angle to another MQTT topic called *wiper_angle_topic*. First 90 degrees, then the script waits for 0.5 s and sends 0 degrees. The sleep is added to be sure that the servo wired to Pico has time to move.

rain_detector_UART_MQTT.sh This script is a link between Pico and the logic working in the other script. First, the script reads data from the "rain_detector_logic.sh". To achieve that, the *mosquitto* library is used to subscribe to a MQTT topic. The subscriber will be running in the background. The data received is piped to the temporary file called *task*. The script checks if there is any new line in the *task* file. If there is a new MQTT message, it prepares the message and sends this data via the serial port to the Pico. Next, the one line of data from the serial port is read. The part of JSON message called "rain_detect" is processed. If it is true, the message 'RAINING' is published via MQTT (topic: *rain_detector_topic*).

2.1.5 Access Point

The RPi is set up to work as an access point. This has been done by using the network manager *nmcli*.

```
$ sudo nmcli con add type WiFi ifname wlan0 mode ap con-name AP ssid "EMLI-TEAM-17"
```

This sets up the WiFi with the interface name *wlan0*. The mode set to ap which means accesspoint and con-name is the name of it, and ssid being what the WiFi is called. Then the AP has been set to use authentication type WPA-PSK and the password set by saying:

```
$ sudo nmcli con modify AP WiFi-sec.key-mgmt wpa-psk
$ sudo nmcli con modify AP WiFi-sec.psk "emliemli"
```

Lastly the IP address is set manually to be static and set to use 2.4 GHz which will be bands bg and channel 6 to align with standards.

```
$ sudo nmcli con modify AP ipv4.method manual ipv4.address 192.168.10.1/24
$ sudo nmcli con modify AP WiFi.band bg
$ sudo nmcli con modify AP WiFi.channel 6
$ sudo nmcli con up AP
```

2.1.6 Local website

For easy access to the images, JSONs and the log file, a local website is hosted on the RPi. This is done by installing Apache2 webserver on the RPi and setting the webserver user as the owner of the data directory "/var/www/html/" and then granting the emli user access. Interactive webpages are made using php files, index.php being the main page. Using the php script it is possible to run bash commands and display the output such as the current time, make hyperlinks to a folder of files such as the images and sidecar files, and hyperlink to a log file so the log can be seen on the webpage. To see the images on the webpage the images is stored in the directory "/var/www/html/images/". The log file is stored as "/var/www/html/log.log".

2.1.7 Power failure recovery

The camera is to be placed in a remote location, probably solar-powered, and may at times deplete the battery. The camera must become fully operational after bootup, by itself. To achieve this, the RPi is set up to make use of services that are enabled on boot up. The following services are running on the camera:

- "externalTrigger.service"
- "motionDetector.service"
- "updateTimeFromDrone.service"
- "updateJsonOfTransferredFiles.service"
- "rainDetector_logic.service"
- "rainDetector_UART_MQTT.service"

Each service calls its respective bash script on bootup, some waiting for specific services to be available before running. An example can be seen in listing 2.1.7. `externalTrigger.service` is waiting for `mosquitto` services to be running before starting an MQTT-dependent service like the external trigger.

Another way of running scripts, even after a reboot, is to make use of cronjobs. These run periodically, which is precisely what is needed for the time trigger of `"take_photo.sh"`, which should be run every 5 minutes. The command that triggers `"take_photo.sh"` as a cronjob is inserted into the `"/etc/crontab"` file as:

```
*/5 * * * * emli /path/to/take_photo.sh Time /var/www/html/images/ true /home/emli/webcam/ /home/emli/webcam.log/
```

The stars correspond to every minute, hour, day, month, year. The first star is divided by 5 which means every 5 minutes it will run the script `take_photo` with the parameters `triggerType`, `imageLocation`, `subfolderWithDate`, `locationOfSymLink1`, `locationOfSymLink2`.

```
1 [Unit]
2 Description=ExternalTrigger MQTT reciever
3 After=mosquitto.service
4 User=emli
5 [Service]
6 Type=simple
7 ExecStart=/home/emli/EMLI-mini-project/src/RPi/Camera/external_trigger.sh
8 [Install]
9 WantedBy=multi-user.target
```

Listing 1: `externalTrigger.service`

Also, the startup of the access point needs to be automated, this is done by writing:

```
$ sudo nmcli connection modify AP connection.autoconnect yes
```

2.1.8 Logging

The logging functionality of the camera is written into the bash scripts using `rsyslog`. In the config `"/etc/rsyslog.conf"` for `syslog`, a line has been added:

```
local7.* /var/www/html/log.log
```

This enables logs using facility `local7` with a log level and a topic identifying the source script. `local7` is a facility open for custom use in the Linux system. In this project the `"info"` log level has been used where messages seemed relevant. All logs written to the `local7` facility are then appended to `"/var/www/html/log.log"`. Examples of this are every time a photo is taken, when it starts and stops raining, and every file synced to a drone. An example of the use from when it stops raining is:

```
logger -p local7.info -t rain_detect "Stopped raining"
```

Here the `-t` parameter is a tag parameter, which is here used to see which service logged this line. The appended line in the log file would then look like this:

```
2024-05-11T12:25:02.073721+02:00 raspberrypi rain_detect: Stopped raining
```

2.1.9 Time synchronization

The time synchronization is done using a script `"get_drone_ip.sh"` which echo the IP of a drone, if one is connected. This is done by checking the stations connected to `wlan0` with `iw`:

```

1 #find mac addresses connected:
2 iw dev wlan0 station dump | grep Station | awk '{print $2}'

```

and then using the mac address found to check the Address Resolution Protocol to find the IP corresponding to said mac address:

```

1 #find IP of mac addresses connected:
2 arp -n | grep $macFound | awk '{print $1}'

```

This IP is then used in another script "update_time.sh":

```

1 ipOfDrone=$(bash /home/emli/EMLI-mini-project/src/RPi/Networking/get_drone_ip.sh)
2 #if not "No drone found"
3 if [ "$ipOfDrone" != "No drone found" ]; then
4     #update the time
5     ntpdate $ipOfDrone
6 fi

```

This sets the time from the NTP server running on the drone.

2.2 Drone

2.2.1 Log WiFi signal

The drone is set up to automatically log the WiFi link quality and signal level to a MySQL database running on the drone. This is done by creating a database and a table in the database:

```

1 CREATE DATABASE emli.WiFi-quality;
2 USE emli.WiFi-quality;
3 CREATE TABLE WiFi_signal (WiFi.linkquality INT, signal_level INT, epoch_time INT);

```

and to append the data to the table from the bash script "save_link_quality.sh", first the link quality and signal level is read from "/proc/net/wireless", piped to get only the part using the interface *wlo1*, taking the word for each value and delimiting this by a "." and only take the first part. The result is inserted into the database:

```

1 link_quality=$(cat /proc/net/wireless | grep wlo1 | awk '{print $3}' | cut -d '.' -f1)
2 signal_level=$(cat /proc/net/wireless | grep wlo1 | awk '{print $4}' | cut -d '.' -f1)
3
4 mysql -h "$MYSQL_HOST" -u "$MYSQL_USER" -p"$MYSQL_PASSWORD" "$MYSQL_DATABASE" \
5     -e "INSERT INTO $MYSQL_TABLE (WiFi.linkquality, signal_level, epoch_time) \
6         VALUES ('$link_quality', '$signal_level', '$time_epoch');"

```

2.2.2 File transfer from Raspberry Pi

The drone is set up with an SSH key to the camera, giving it access to the RPi file system. When the drone is connected to the access point, it runs the following *rsync* command.

```

rsync --progress --verbose --remove-source-files -avzL -e "ssh -i ~/.ssh/id_ed25519_rpi" emli@192
.168.10.1:/home/emli/webcam/ /drone/directory/files

```

The file structure of the photos on the camera, is as follows

```

/var
  /www
    /html
      /images
        /2024-05-11
        /2024-05-12

```

```
/home
  /emli
    /webcam
      /2024-05-11
      /2024-05-12
    /webcam_log
      /2024-05-11
      /2024-05-12
```

The local website directory `/var/www/html/images/` contains all the files created by the camera. The other two directories, `webcam` and `webcam_log` are used for synchronizing with the drone. These contain symbolic links to every original file in `/var/www/html/images/`. The `webcam` directory is the directory from which the drone copies files, which are then deleted by the `rsync` call when transferred because of the parameter `--remove-source-files`. The option `L` means that the source files of symbolic links are copied. The `updateJsonOfTransferredFiles` service awaits a difference using the `diff` command to check the `webcam` directory and the `webcam_log` directory. When something is only in the `webcam_log` directory the differences stem from the files having been synchronized to a drone. The sidecar files belonging to the found differences are then updated in the website directory, with information about the time of synchronization and the ID of the latest connected drone. The symbolic link in `webcam_log` of the files that were found to be the difference is then deleted from the `webcam_log` directory. This way, we get the benefits of fast and secure data transfers over `rsync`, while keeping track of what files have been transferred, in case of loss of connection while transferring. The reason for this solution was that if a disconnect happened before all the files were synchronized to the drone, the only way to check what had been sent was to check the log of `rsync`. But in case of a disconnect, `rsync` thought it had sent a bunch of files that were never received on the drone. Then every time the file has been received by the drone the symbolic link would be deleted, making sure every photo eventually will be synced. The symbolic links avoid redundancy of data in the file system which means saving space on the SD-card and time for writing to memory.

2.3 Cloud

As explained previously the drone downloads files from the RPi Wildlife cameras when connected. An image along with its sidecar file (`.jpg` + `.json`) are stored in the drone's memory. After landing, the files are supposed to be copied manually to the Cloud offloading folder. Then the annotation script `animal_recognition.sh`, is run. There are two input parameters, the directory of the input files and the git repository directory which will be the output directory. The annotation is done using the LLM model in *Ollama* called *LLaVA*: Large Language and Vision Assistant. By default, it is set up to use the 7b version. The script runs the *LLaVA* model in the background to be able to use it with API calls and not spin it up for each image. To run it in the background an `&` sign is added in the end of the run line:

```
ollama run llava:7b &
```

Then in the `perform_annotation()` function, the image is prepared for the *ollama* API. It is required to convert them into ASCII text using Base64 encoding scheme. Then using the API call to the *LLaVA* model^[1]:

```
curl http://localhost:11434/api/generate -d '{ "model": "llava", "prompt": "Describe the picture in one sentence", "images": [ "$img64" ] }
```

If the API call gives a valid output, the output which consist of many JSONs, one for each word replied, is converted with `jq` to be easy to read. This is done by piping the output into

jq and looking for the response variable, then piping the answer of this to remove all newline characters:

```
echo $output | jq -r '.response' | tr -d '\n'
```

The *final_output* is added to JSON file by "multiplying" the two JSON and, saving the result in temp.json. Then moving temp.json into the current JSON file and thereby adding the annotation to the original file:

```
json_annotation="{\"Annotation\": {\"Source\": \"Ollama:$ollama_model_version\", \"Test\": \"\n    $final_output\n\"}}\"\njq -s '[0] * [1]' <(echo \"$json_annotation\") \"$json_file\" > temp.json && mv temp.json \"$json_file\"
```

The *perform_annotation()* function is run on every .jpg file found in the specified directory. After all annotations are done, the script moves all the files to the GitHub repository folder and pushes all .json files. To do so, the tool "push_json.sh" script is run. There was a requirement to push only JSON files, not the .jpg. We set this up in two steps. The first one is applied in the "push_json.sh" script. Only .json files are added to the staging area. Then the script checks if there is anything to commit. If there is, the commit is performed and the .json files are pushed.

The second way to make sure that we won't have any .jpg files pushed is to create a .gitignore file in the repository's main folder and added the following ignore: "*.jpg"

2.4 Security

The RPi is set up with multiple layers of security.

The access point is the first layer of security, where a password is needed to connect. As mentioned in section 2.1.5 the authentication type for the WiFi is set to WPA-PSK. To make sure the WiFi access point is more secure, WPA1 and WPA2-TKIP which are both considered insecure, have been disabled. This is done by writing:

```
$ sudo nmcli con modify AP 802-11-wireless-security.proto rsn\n$ sudo nmcli con modify AP 802-11-wireless-security.pairwise ccmp
```

If someone gets through the WiFi password and is connected to the access point. The next layer of security is using *nftables*, which is set up as a firewall to filter network packages. This is configured to only allow ports used for SSH and HTTP, closing for exploits in parts of the system that are not in use. This is done running the following script:

```
1  #!/sbin/nft -f\n2  flush ruleset\n3  table ip filter {\n4      chain input {\n5          type filter hook input priority 0;\n6          # Allow SSH (TCP port 22)\n7          tcp dport 22 accept\n8          # Allow HTTP (TCP port 80)\n9          tcp dport 80 accept\n10         # Drop all other incoming traffic\n11         drop\n12     }\n13 }
```

This means that if someone unauthorized is connected they will be able to only see the website with the images and the log file and try to connect over SSH.

The next layer of security is on the SSH connection to the RPi. This is protected against brute force attacks using *fail2ban*, giving three attempts before banning the IP address that fails the login. This is done by editing the config "/etc/fail2ban/jail.local" to the following:

```
1 bantime = 1h
2 findtime = 1h
3 maxretry = 3
```

This means that on 3 failed connection tries within the *findtime* of 1 hour, the IP will be banned for 1 hour.

3 Test and results

The focus of testing has primarily been on verifying the stability of the system. Stability has been tested by restarting the system and looking at the status of the services running after boot up. All services run on boot up and work as intended.

The testing is shown in the video handed in together with this report.

To test the photos and making of sidecar files, the external trigger button has been pressed, moved stuff in front of the camera and looking at what happens every fifth minute and then looking at the log and the files through the webserver. From this the result is that it works as intended, except there being a race condition to connect to the camera. If two photos happen at the same time, the "take_photo.sh" script is run while the camera is not available, it will exit with status 1, error.

The rain detector and wiper is tested by pushing the button simulating the rain detection sensor and seeing the servo moving back and forth as long as the button is being held

The time synchronization of the camera is tested, by connecting it to a laptop without internet access and an NTP server running. The time of the camera was set to another date, and the moment the laptop was connected, the time updated to the current time of the test.

Handling of loss of connection during a *rsync* command has been tested by disconnecting the drone from the access point of the camera before all files were transferred. There was no loss of files in the process, and all files that were transferred were correctly annotated. Also the WiFi link quality in the MySQL database is updated every second while the drone is connected.

The *Ollama* annotations were tested on a small set of images found online of different animals on the savanna, together with some sidecar files created by the RPi. This has been pushed to the GitHub under "images_to_cloud/2024-05-11/", where the results can be seen.

4 Conclusion

This project has implemented a RPi as a wildlife camera with an access point. Software has been developed for obtaining data from the camera to a drone, implemented LLM for annotating the images, and saving the camera data to *github*.

The camera makes use of embedded systems, namely the Pico and ESP8266, to sense and interact with its environment. Bash scripts have been made use of as the main programming language for the project, running them with services available on the Linux system, such as *systemd* and *cron*, which also ensures operability, even after a reboot. The safety of the RPi is enhanced by use of *ssh* and *fail2ban* for login and *nftables* to close unused ports on the RPi. The programs on the camera log to a facility in the Linux system, and is easily extendable.

The Drone runs a NTP server, that the RPi can synchronize time with. It knows the access point of the RPi and connects automatically when it is available. The drone has an *ssh* key to the RPi and synchronizes files using *rsync* over WiFi, the link quality is logged to a MySQL database for flight path optimization.

The cloud runs *Ollama* for annotation on the images and saves the JSON sidecar files to GitHub.